HEMISPHERIC

AI Platform Engineer - Exercise 1

Data Infrastructure for AI Training

Goal

Using this exercise, we would like to learn about your software engineering and programming skills, in the type of tasks you'll see in this role.

Overview

In this exercise you'll be writing a Python data provider for a deep-learning training process and the orchestrator for launching training jobs.

Our data factory collects EEG data from different subjects across one or more visits to our labs. Alongside the EEG data, some metadata is collected including gender, age and so on. The data provider is aimed to go over a filtered section of the data, randomly select a sample of EEG data and send it to the training process (the data consumer). The deep-learning team will use the orchestrator that you've built to run their training jobs on the filtered data they choose.

You are given the following:

1. 100 files containing EEG data, sampled at 1KHz. Each sample contains 4 channels of float32.
2. 100 matching JSON files containing the metadata about the EEG files, including information such as name, gender and age.
3. A consumer code that emulates a deep-learning job. It'll connect via TCP to the list of ports provided, read the data from those ports and log the training progress. The consumer is expecting chunks of 10 seconds of data.
4. A README.md file detailing how to build and run the consumer.

The zip file containing all of the above can be downloaded from here.

Requirements

1. Write a tool that orchestrates the training process.
2. The orchestrator gets command line arguments in order to train the model based on a certain filter of the data. For instance: python run_training.py --gender female --age >20. You can come up with a different syntax for this, of course.

Proprietary & Confidential. Copyright © 2026 HEMISPHERIC

3. Write the data provider process. It should read the data that matches the filters, start a TCP server on the desired ports, and send all of the data as random chunks of 10 seconds.
    1. Each chunk should start with the visit ID (it's a UUIDv4, so 128bit) followed by the float32 array.
    2. If a certain file contains 60 seconds of data, it could be looked at as 6 chunks of 10 seconds. The order in which those chunks are sent - is random.
4. Design your architecture (but don't implement) such that the producers and consumers could run in a cluster.

Deliverables

1. Your complete code, in a form we can run easily.
2. Instructions on how to run your project.
3. Readme.
4. All deliverables must be uploaded to a private git repository, shared with us.

Pay attention to

1. Clean, readable and maintainable code.
2. Should you decide to use a coding agent, please include a markdown file with the most important prompts used.

Feel free to ask us any questions.

Good luck!

Proprietary & Confidential. Copyright © 2026 HEMISPHERIC